

Universidad Católica Boliviana “San Pablo”



UNIVERSIDAD
CATÓLICA
BOLIVIANA

Microservicio WebSocket para Conteo de Usuarios

Docente: Ing. Miguel Pacheco

Estudiante: Leonardo Delgado Medrano

Semestre: I – 2025

Tecnologías Web II

1. Análisis del Proyecto Integrador

- **Identificación clara del endpoint**

Endpoints que consumiremos:

- **POST** <http://localhost:3000/login>

Para obtener el token JWT de autenticación

- **GET** <http://localhost:3000/cuentas-por-rol/:rol>

Para obtener la cantidad de usuarios que se tiene dependiendo de cada rol en el sistema de gestión de taller de grado.

- **Justificación de la elección del endpoint**

Los endpoints permiten obtener datos esenciales para el conteo en tiempo real.

WebSocket es ideal para notificaciones instantáneas de cambios en los registros.

- **Descripción técnica del endpoint (estructura, método, formato de datos)**

Método	URL	Formato de datos	Formato de respuesta	Autenticación
POST	/login	JSON(usuario, contraseniaUser)	JSON	-----
GET	/cuentas-por-rol/:rol	-----	JSON	JST (Bearer token)

2. Diseño del Microservicio

- **Objetivo del microservicio claramente definido**

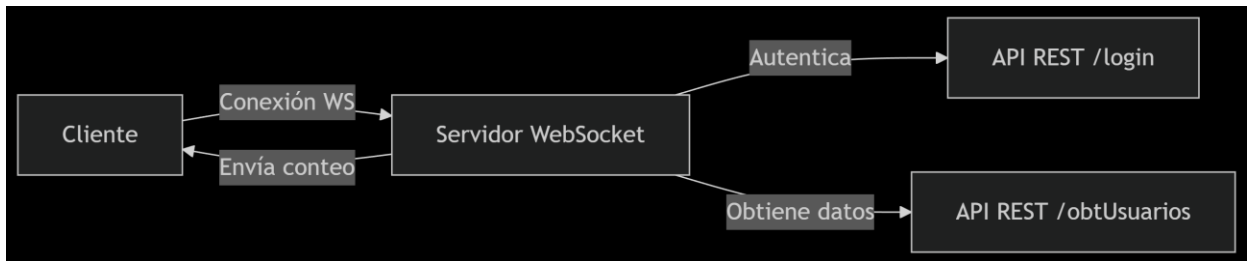
Construir un microservicio WebSocket que cuente y notifique en tiempo real la cantidad de usuarios por rol (directores, docentes, estudiantes), consumiendo la API del proyecto integrador.

- **Elección y justificación de la tecnología de comunicación**

WebSocket: Elegido por su capacidad de comunicación bidireccional y baja latencia.

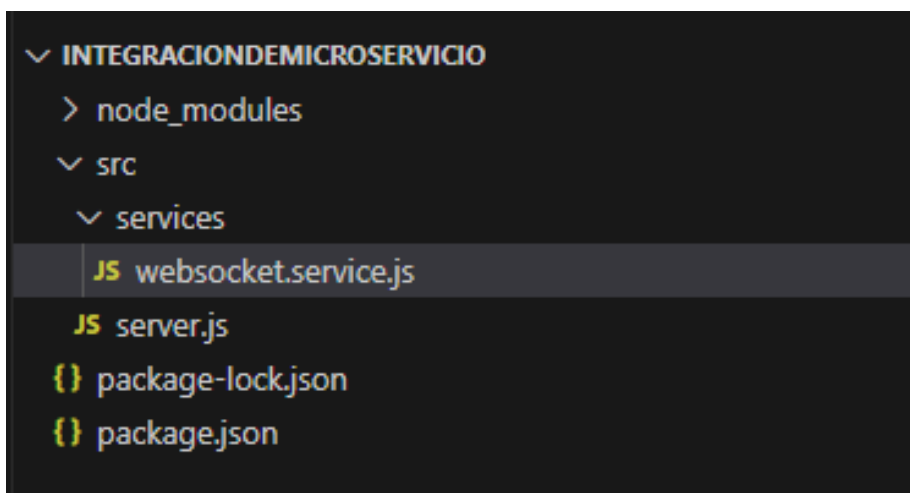
Node.js + ws: Librería ligera para implementación eficiente.

- Diagrama del flujo de integración



3. Implementación Técnica

- Estructura del proyecto



- Código base

1. WebSocket.service.js

```
import { WebSocketServer } from 'ws';
import axios from 'axios';

let wss;
let authCredentials = {}; // Almacena credenciales por conexión

export function initWebSocket(server) {
  wss = new WebSocketServer({ server });

  wss.on('connection', (ws) => {
    console.log('Cliente WebSocket conectado');

    // Esperar el primer mensaje con las credenciales
    ws.once('message', async (message) => {
```

```

    try {
      const { usuario, contrasenia } = JSON.parse(message);

      if (!usuario || !contrasenia) {
        throw new Error('Credenciales faltantes');
      }

      // Guardar credenciales para esta conexión
      authCredentials[ws.id] = { usuario, contrasenia };

      // Enviar conteo inicial
      await sendUserCount(ws);

      // Configurar intervalo para actualizaciones
      const interval = setInterval(() => sendUserCount(ws), 10000);

      ws.on('close', () => {
        clearInterval(interval);
        delete authCredentials[ws.id];
        console.log('Cliente desconectado');
      });

    } catch (error) {
      console.error('Error en autenticación:', error.message);
      ws.send(JSON.stringify({ error: 'Autenticación fallida' }));
      ws.close();
    }
  });

  // Agregar ID único a la conexión
  ws.id = Math.random().toString(36).substring(7);
});
}

async function sendUserCount(ws) {
  try {
    const { usuario, contrasenia } = authCredentials[ws.id] || {};
    if (!usuario || !contrasenia) return;

    const conteo = await contarUsuarios(usuario, contrasenia);
    if (conteo) {
      ws.send(JSON.stringify(conteo));
    }
  } catch (error) {
    console.error('Error al enviar conteo:', error.message);
  }
}

```

```

    }
  }

  async function contarUsuarios(usuarioo, contrasenia) {
    try {
      // 1. Autenticación
      const loginResponse = await axios.post(`http://localhost:3000/login`, {
        usuario: usuarioo,
        contraseniaUser: contrasenia
      });

      const token = loginResponse.data.token;

      // 2. Obtener conteos
      const requests = [1, 2, 3].map(rol =>
        axios.get(`http://localhost:3000/cuentas-por-rol/${rol}`, {
          headers: { Authorization: `Bearer ${token}` }
        })
      );

      const [directores, docentes, estudiantes] = await Promise.all(requests);

      return {
        directores: directores.data.count,
        docentes: docentes.data.count,
        estudiantes: estudiantes.data.count,
        total: directores.data.count + docentes.data.count + estudiantes.data.count
      };
    } catch (error) {
      console.error('Error en contarUsuarios:', error.message);
      throw error;
    }
  }
}

```

2. server.js

```

// server.js
import http from 'http';
import express from 'express';
import { initWebSocket } from './services/websocket.service.js';

const app = express();
const server = http.createServer(app);

```

```
// Servir el cliente WebSocket para pruebas
app.use(express.static('.')); // sirve client.html desde el root

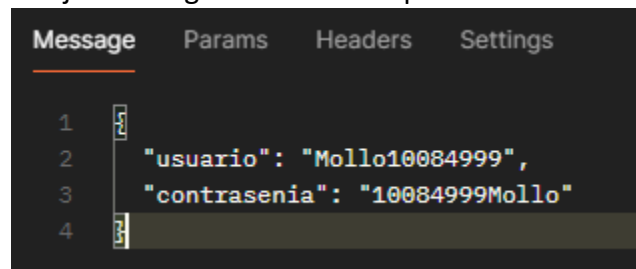
// Iniciar WebSocket
initWebSocket(server);

// Iniciar el servidor
const PORT = 8080;
server.listen(PORT, () => {
  console.log(`Servidor HTTP y WebSocket escuchando en
http://localhost:${PORT}`);
});
```

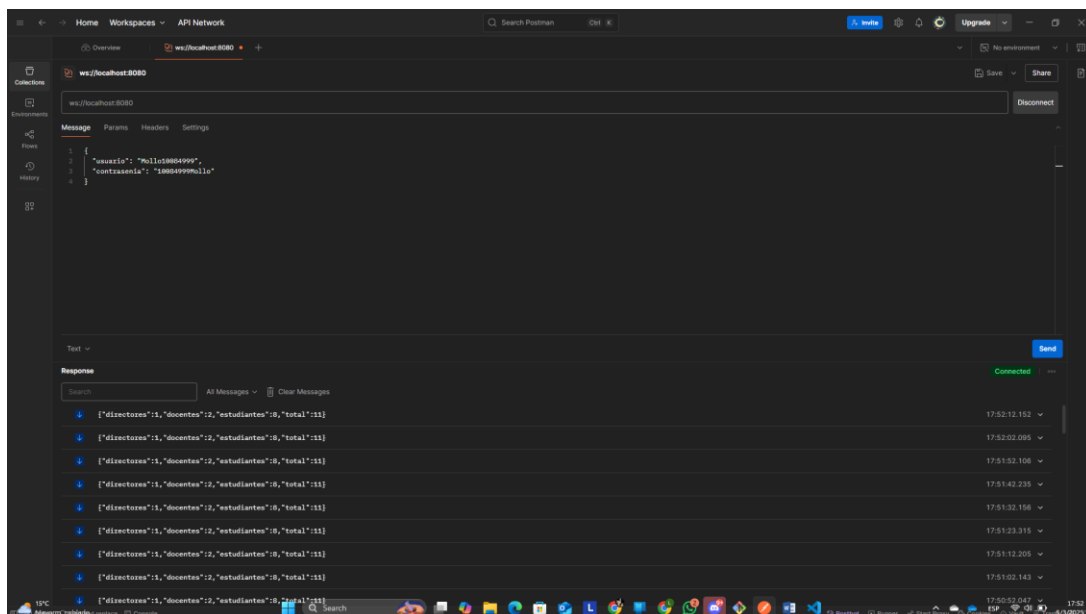
4. Pruebas y Documentación

- Evidencia

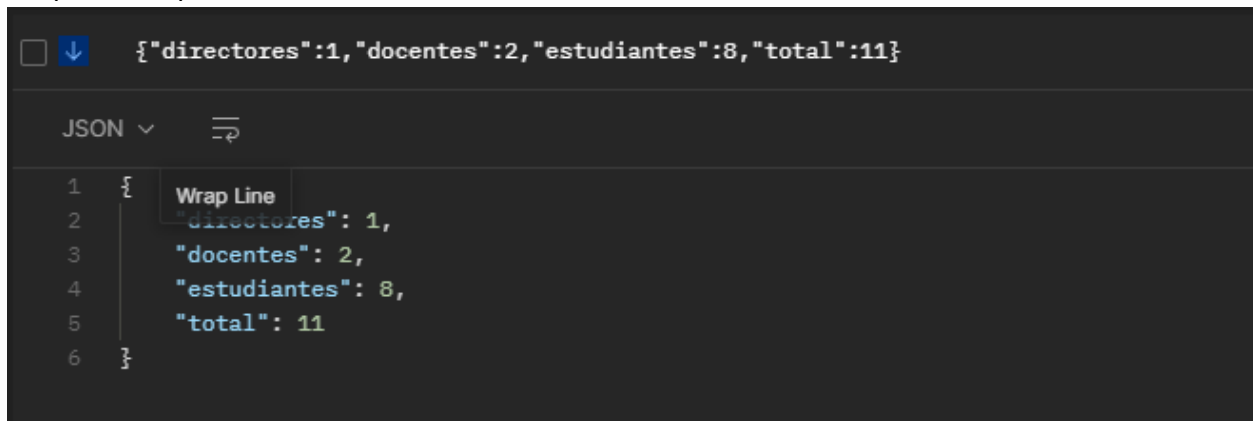
Prueba en postman, con el switch request type WebSocket, requiere de que se envíe un mensaje en el siguiente formato para la autenticación:



Prueba de que conecta exitosamente con el microservicio:



Respuesta esperada en formato JSON:



```
{\"directores\":1,\"docentes\":2,\"estudiantes\":8,\"total\":11}
```

- **Pruebas manuales**

- Conexión exitosa al WebSocket.
- Validación de actualización automática cada 10 segundos.

5. Anexos

Enlace del repositorio del backend en github:

<https://github.com/LEONGO037/BackendTallerDeGrado>

Enlace del repositorio del microservicio en github:

<https://github.com/LEONGO037/IntegracionDeMicroservicio>